

SV Engine: Mathematical Design

vsrg-analysis Player

TL;DR. *Time-space* is smooth: notes scroll at a velocity that’s a function of wall-clock time, and the cumulative position is a continuous function. *Beat-space* is not smooth, because charts can have *stops* (time passes, beats don’t) and *warps* (beats jump, time doesn’t) — and a warp creates a literal jump in the cumulative-position function that no time-space velocity can reproduce.

Came up by Yucky, written up by a **CLANKER**. If you want a even simpler explanation, think about it like the difference between an indestructible ball and a destructible ball that is usually round but sometimes not (make a PR if you still don’t understand this)

The rest of this document formalizes that observation: time-space is integration against ordinary Lebesgue measure $d\tau$; beat-space is integration against a *Stieltjes measure* dB that allows point masses (= warps). Once you write it this way, “which game’s SV are we using” collapses to “which measure are we integrating against,” and switching engines at runtime is just swapping μ on the chart document. One integrator, multiple measures, one codebase.

Abstract

This document gives the mathematical foundation for the unified scroll-velocity (SV) framework used by the Player. The framework is built on a single *measure-theoretic primitive*: render distance is a Lebesgue–Stieltjes integral of a velocity against a chart-derived measure on chart time. Engines differ only in their choice of measure; the document carries the integrand and the measure as orthogonal data.

Three structural facts drive the design. First, time-space and beat-space are not different positioning theories but two choices of measure — Lebesgue $d\tau$ versus the Stieltjes measure dB induced by the timing map. Second, the failure of time-space to represent warps is a property of the *measure*: $L^1(T, d\tau)$ functions cannot place mass at a point, so no enrichment of the integrand alone can recover warps under Lebesgue measure. Third, conversion between measures is exact wherever the timing map is absolutely continuous, by Radon–Nikodym; warps are precisely the failure of absolute continuity.

The engine architecture follows: the canonical document carries a measure $d\mu$ and an $L^1(T, d\mu)$ integrand v . A single integrator computes $\int v d\mu$. Engine swap reduces to swapping the measure on the document; the integrator is unchanged. Per-column SV is K integrands against one shared measure; per-note SV is sparse-indexed integrands against the same measure.

Contents

1	Function spaces and measures	1
2	The render-distance functional	2

3	Instantiations	3
3.1	Time-space (osu!mania, Quaver)	3
3.2	Beat-space (Etterna XMOD)	3
3.3	Constant-BPM (IIDX)	3
3.4	Identity	4
4	Lossless conversion	4
5	Stops, delays, and warps	5
5.1	Stops and delays	5
5.2	Warps	5
6	BPM–SV coupling toggle	6
7	Per-column and per-note SV	6
8	Numerical exactness	6
9	Architectural mapping	7
9.1	The integrator	7
9.2	SVDData	7
9.3	Engines as measure choices	8
9.4	Engine swap	8
10	Per-frame predictor (cull-space)	8
10.1	The anchor-and-extrapolate algorithm	9
10.2	Re-anchoring on breakpoints	9
10.3	Re-anchoring on audio callbacks and discontinuities	9
10.4	Eased segments	10
10.5	Performance and jitter trade	10
10.6	Native-port boundary	10
11	Open extensions	11

1 Function spaces and measures

We work on a closed chart-time interval $T = [t_0, t_{\text{end}}] \subset \mathbb{R}$. The framework is parameterized by a pair (μ, v) where μ is a positive Radon measure on T and v is a μ -integrable function. The render distance is $\int v d\mu$. The two “spaces” the rhythm-game literature distinguishes correspond to two choices of μ .

Definition 1 (Time-space). The time-space framework is the pair $(d\tau, v)$ with $v \in L^1(T, d\tau)$ and $d\tau$ Lebesgue measure on T .

Definition 2 (Beat-space). A *timing map* is a non-decreasing right-continuous function $B : T \rightarrow \mathbb{R}_{\geq 0}$ of bounded variation, with $B(t_0) = 0$. The *Stieltjes measure* dB is the unique Radon measure on T with

$$dB([\alpha, \beta]) = B(\beta) - B(\alpha^-) \quad \text{for } \alpha \leq \beta \in T.$$

The beat-space framework is the pair $(dB, s \circ B)$ with $s : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ measurable and $s \circ B \in L^1(T, dB)$.

Remark 3 (Anatomy of dB). By the Lebesgue decomposition, every Radon measure on T splits as $dB = (dB)_{AC} + (dB)_{sing}$, where $(dB)_{AC} \ll d\tau$ has a Radon–Nikodym density $B'(\tau) d\tau$ and $(dB)_{sing} \perp d\tau$ is supported on a Lebesgue-null set. For chart timing maps the singular part is purely atomic: it is a finite sum of point masses, one at each warp:

$$dB = B'(\tau) d\tau + \sum_w \Delta b_w \cdot \delta_{\tau_w}. \quad (1)$$

The atoms encode warps; the absolutely continuous part encodes BPM. Stops contribute to $(dB)_{AC}$ via $B' = 0$ on the stop interval — they are absolutely continuous, just degenerate.

Definition 4 (Position-dependent zoom). The *zoom* $z : T \rightarrow \mathbb{R}_{>0}$ is bounded measurable in chart time and is applied multiplicatively at the playhead, outside the integral. Time-space engines fix $z \equiv 1$; Etterna’s #SPEEDS produces a nontrivial z .

Definition 5 (Base BPM). $BPM_0 := BPM(t_0)$ is the BPM of the first segment.

Definition 6 (Column index). Let $C = \{0, \dots, K-1\}$ index the playfield columns. Where the integrand varies per column we write $v(\tau, c)$ and $z(\tau, c)$. The measure μ is column-independent: it is a property of the chart timing data only.

2 The render-distance functional

Definition 7 (Render distance). Given a measure μ on T , an integrand $v \in L^1(T, \mu)$, and a zoom z , the *render distance* between chart times $a, b \in T$ as observed from playhead a is

$$d_\mu(a, b) := z(a) \cdot \int_{(a, b]} v(\tau) d\mu(\tau). \quad (2)$$

We use half-open intervals $(a, b]$ throughout to make point-mass attribution unambiguous: a warp at τ_w contributes to $d_\mu(a, b)$ iff $\tau_w \in (a, b]$.

Definition 8 (Cumulative function). The *cumulative function* is

$$C_\mu(t) := \int_{(t_0, t]} v(\tau) d\mu(\tau). \quad (3)$$

Proposition 9 (Cumulative decomposition). *For all $a, b \in T$,*

$$d_\mu(a, b) = z(a) \cdot (C_\mu(b) - C_\mu(a)).$$

When $z \equiv 1$, C_μ alone determines render distance; cull-space queries reduce to a single bisection on a precomputed array of C_μ samples.

Remark 10 (Why C is cached and z is per-frame). C_μ depends only on the chart, so it is computed once at chart load and queried at $O(\log n)$ per call. $z(a)$ depends on the playhead and is evaluated per frame. Splitting them lets the cull-space window be a single bisect over C_μ , with z applied as a uniform multiplier on the visible delta. This is exactly the structure the existing **SVEngine** Protocol exposes (`cumulative_at`, `render_multiplier_at`, `distance`).

Remark 11 (Cull-space approximation under nontrivial z). When z varies, $z(a) \cdot (C_\mu(b) - C_\mu(a))$ approximates the true integral by treating z as constant over $(a, b]$. The error during a brief z -transition is at most one segment’s multiplier, which the Player pads for. This matches Etterna’s ArrowEffects.cpp XMOD branch.

3 Instantiations

3.1 Time-space (osu!mania, Quaver)

$\mu = d\tau$, $z \equiv 1$:

$$d_{d\tau}(a, b) = \int_{(a, b]} v(\tau) d\tau.$$

The timing map B does not appear. Stops and warps are not natively expressible (§5).

3.2 Beat-space (Etterna XMOD)

$\mu = dB$, integrand $v(\tau) = s(B(\tau))$, $z(a) = \text{speed_percent}(a)$. By definition,

$$d_{dB}(a, b) = z(a) \cdot \int_{(a, b]} s(B(\tau)) dB(\tau). \quad (4)$$

On any interval where B is absolutely continuous, $dB = B'(\tau) d\tau$ and (4) reduces to a Lebesgue integral. Substituting $b = B(\tau)$ when B is AC and strictly increasing,

$$\int_{(a, b]} s(B(\tau)) B'(\tau) d\tau = \int_{B(a)}^{B(b)} s(\beta) d\beta = D(B(b)) - D(B(a)),$$

where $D(\beta) = \int_0^\beta s$ is the displayed-beat function. Converting to render seconds via the base BPM:

$$d_{dB}(a, b) = z(a) \cdot \frac{60}{\text{BPM}_0} \cdot (D(B(b)) - D(B(a))) \quad (\text{AC region}).$$

This is exactly Etterna's XMOD positioning equation. At a warp $\tau_w \in (a, b]$, the singular part of dB contributes an additional point-mass term $s(b_w^+) \cdot \Delta b_w$ via (1), which the AC formulation above does not capture.

3.3 Constant-BPM (IIDX)

$\mu = dB$, $v \equiv 1$, $z \equiv h$ for player-chosen hi-speed h :

$$d_{dB}(a, b) = h \cdot dB((a, b]) = h \cdot (B(b) - B(a)).$$

Chart SV is dropped; BPM changes propagate through B' and warps through the atoms of dB , but the constant integrand neutralizes any chart-authored multiplier. Floating hi-speed is a separate solve for h at chart load and lives outside the integrator.

3.4 Identity

$\mu = d\tau$, $v \equiv 1$, $z \equiv 1$: $d(a, b) = b - a$.

4 Lossless conversion

The conversion between time-space and beat-space is the Radon–Nikodym relationship between the two measures, valid wherever it applies.

Theorem 12 (Equivalence on AC intervals). *Let $I \subseteq T$ be an interval on which B is absolutely continuous (no warps in I ; stops are permitted). Let $s \in L^1(B(I), d\beta)$ be a beat-space integrand. Define*

$$v^*(\tau) := s(B(\tau)) \cdot \frac{\text{BPM}(\tau)}{\text{BPM}_0}, \quad \tau \in I. \quad (5)$$

Then for all $a, b \in I$,

$$\int_{(a,b]} v^*(\tau) d\tau = \frac{60}{\text{BPM}_0} \int_{(a,b]} s(B(\tau)) dB(\tau).$$

That is, the time-space integral of v^ against $d\tau$ equals the beat-space integral of $s \circ B$ against dB , up to the global factor $60/\text{BPM}_0$.*

Proof. On I , B is AC, so by Radon–Nikodym $dB = B'(\tau) d\tau$ with $B' \in L^1(I, d\tau)$, and $B'(\tau) = \text{BPM}(\tau)/60$ a.e. Then

$$\int_{(a,b]} s(B(\tau)) dB(\tau) = \int_{(a,b]} s(B(\tau)) B'(\tau) d\tau = \int_{(a,b]} s(B(\tau)) \cdot \frac{\text{BPM}(\tau)}{60} d\tau.$$

Multiplying by $60/\text{BPM}_0$ gives the right-hand side equal to $\int_{(a,b]} s(B(\tau)) \cdot \text{BPM}(\tau)/\text{BPM}_0 d\tau = \int_{(a,b]} v^*(\tau) d\tau$. $\square$

Theorem 13 (Time $\rightarrow$ beat). *Under the same hypotheses as Theorem 12, given a time-space integrand $v \in L^1(I, d\tau)$, define*

$$s^*(\beta) := v(B^{-1}(\beta)) \cdot \frac{\text{BPM}_0}{\text{BPM}(B^{-1}(\beta))} \quad \text{for } dB\text{-a.e. } \beta \in B(I). \quad (6)$$

Then $\int_{(a,b]} v(\tau) d\tau = (60/\text{BPM}_0) \int_{(a,b]} s^(B(\tau)) dB(\tau)$.*

Proof. Symmetric to Theorem 12. B^{-1} is well-defined dB -a.e. on $B(I)$: stops are atoms in the $d\tau$ -pushforward but dB -null in beat space (stops are intervals where B is constant, so dB assigns them measure zero), so the failure of pointwise invertibility on stops occurs on a dB -null set and does not affect the integral. $\square$

Remark 14 (Where conversion fails). Theorem 12 requires B absolutely continuous on I . Warps violate AC: at a warp τ_w , B has a jump $B(\tau_w) - B(\tau_w^-) = \Delta b_w > 0$, so dB has an atom $\Delta b_w \cdot \delta_{\tau_w}$ that cannot be written as $B'(\tau) d\tau$ for any $B' \in L^1$. The conversion theorem does not extend across warps; §5 treats this as an irreducible obstruction.

Corollary 15 (Zoom is orthogonal). *The zoom $z(a)$ is multiplied outside the integral and does not appear in (5) or (6). Conversion preserves z verbatim; the engine independently chooses whether to honor it.*

Remark 16 (Distance preserved; cumulative not). Theorems 12 and 13 preserve $d_\mu(a, b)$ for all a, b . The cumulative C_μ is *not* preserved pointwise across a change of measure: an antiderivative is measure-specific. Engine swap must rebuild C_μ from scratch in the new measure; cached values are not translatable.

5 Stops, delays, and warps

5.1 Stops and delays

A stop at chart time τ_s of duration δ has B constant on $[\tau_s, \tau_s + \delta]$. Then dB assigns this interval measure zero, so

$$\int_{(\tau_s, \tau_s + \delta]} s(B(\tau)) dB(\tau) = 0$$

for any integrand — notes do not move. In time-space, the analogous behavior requires setting $v = 0$ on the stop and re-parameterizing the chart-time domain by $+\delta$ for all subsequent events. This is a domain extension, not a pointwise patch.

5.2 Warps

A warp at τ_w of beat extent $\Delta b > 0$ contributes the atom $\Delta b \cdot \delta_{\tau_w}$ to dB . Then

$$\int_{(a,b]} s(B(\tau)) dB(\tau) = \int_{(a,b] \setminus \{\tau_w\}} s \cdot B' d\tau + s(b_w^+) \cdot \Delta b \cdot \mathbf{1}_{\tau_w \in (a,b]}.$$

The atom contributes a step jump in C_{dB} at τ_w with size proportional to the SCROLLS ratio at the warp’s landing beat.

Proposition 17 (Warp irreducibility). *There is no $v \in L^1(T, d\tau)$ such that $\int_{(a,b]} v(\tau) d\tau$ has a jump discontinuity in b at some $\tau_w \in (a, b]$.*

Proof. For $v \in L^1(T, d\tau)$, the cumulative $F(b) = \int_{(t_0, b]} v(\tau) d\tau$ is absolutely continuous in b , hence continuous. No L^1 density against Lebesgue measure produces jump discontinuities in its antiderivative. $\square$

Remark 18 (The obstruction is the measure, not the integrand). Proposition 17 is stronger than “you’d need a Dirac.” Even allowing v to range over signed Radon measures rather than L^1 functions does not help *within time-space*: if one extends the integrand to a measure $v d\tau \rightarrow \nu$, then $\int_{(a,b]} \nu$ depends on ν , not on $d\tau$; one has effectively replaced the time-space pair $(d\tau, v)$ with $(\nu, 1)$, which is a beat-space integral against a different measure. Thus warps are not expressible by enriching the integrand within the time-space framework: they require a different measure.

Corollary 19 (Canonical form). *Any framework intended to support warps must integrate against a measure with nontrivial singular part. The canonical **SVD** **data** stores dB with explicit warp atoms, making beat-space the structurally complete representation. Time-space-authored charts (*osu!*, *Quaver*) embed by setting the singular part to zero.*

6 BPM–SV coupling toggle

Some authoring formats (Quaver) expose a flag controlling whether timing-point BPM scales SVs (**BPMDoesNotAffectScrollVelocity**). In the present framework this is not a separate space: it is a choice of which integrand the chart authored.

Definition 20 (Coupled and decoupled time-space). The *BPM-coupled* time-space integral is $\int v(\tau) \cdot \text{BPM}(\tau) / \text{BPM}_0 d\tau$; the *BPM-decoupled* integral is the plain $\int v(\tau) d\tau$.

Proposition 21 (Coupling is a choice of authoring frame). *A BPM-coupled time-space integrand v_{coup} produces the same render distance as a beat-space integrand s on AC intervals iff $v_{\text{coup}} = s \circ B$ (after multiplying by $60/\text{BPM}_0$).*

Proof. Direct from Theorem 12 with $v^* = (s \circ B) \cdot \text{BPM}/\text{BPM}_0$. $\square$

The flag therefore declares which authoring frame the file uses; the canonical document stores the equivalent decoupled form, and the engine applies coupling at evaluation if requested by the chart’s flag.

7 Per-column and per-note SV

Definition 22 (Selector). A *selector* is an element of $\text{All} \sqcup 2^C \sqcup 2^{\mathcal{N}}$ (All, a column subset, or a note-id subset).

Each velocity record carries a selector. The per-column integrand is

$$v(\tau, c) = m_k \quad \text{where } k = \max\{i : \tau_i \leq \tau, c \in \text{sel}(i)\}.$$

Theorem 23 (Per-column independence). *Fix a measure μ on T and a per-column integrand $v(\tau, c)$ with $v(\cdot, c) \in L^1(T, \mu)$ for each c . Define $d_c(a, b) = z(a, c) \cdot \int_{[a, b]} v(\tau, c) d\mu(\tau)$ and $C_c(t) = \int_{(t_0, t]} v(\tau, c) d\mu(\tau)$. Then §2–§4 apply column-wise: the conversion of Theorem 12 preserves d_c for each c independently, and stop/warp behavior under $\mu = dB$ is per-column identical.*

Proof. μ is column-independent and the analyses of §2–§4 depend on v only through its $L^1(T, \mu)$ membership. Apply column-wise. $\square$

Remark 24 (Storage and per-note). The cumulative cache becomes K parallel arrays $C_0, \dots, C_{K-1}$. Records with selector All share segments structurally; per-column records branch the affected columns. Per-note SV (selector $N \subseteq \mathcal{N}$) is type-compatible: the integrand becomes $v(\tau, n)$ indexed by note. Sparse storage is required for efficient $|\mathcal{N}|$ -many cumulative arrays; the math (Theorem 23) extends without change.

8 Numerical exactness

Proposition 25 (Exactness for piecewise-constant integrands). *If v is piecewise-constant on a finite partition of T and μ is a finite sum of an AC part with piecewise-constant density plus finitely many atoms, then C_μ is piecewise-linear-with-jumps on the common refinement of the partitions. Evaluation of $C_\mu(t)$ at any t reduces to one bisection plus a constant-time arithmetic expression*

$$C_\mu(t) = C_\mu(\tau_k) + (t - \tau_k) \cdot v_k \cdot \rho_k + \sum_{w: \tau_w \in (\tau_k, t]} v(\tau_w^+) \cdot a_w,$$

where ρ_k is the AC density of μ on segment k and a_w is the atomic mass at warp τ_w . No quadrature is required.

Remark 26 (Test invariant). For any SVData and any sample time t , the integrator’s $C_\mu(t)$ must agree with the brute-force reference

$$C_{\text{ref}}(t) = \sum_i v_i \cdot \mu(I_i \cap (t_0, t])$$

to numerical precision (modulo summation order). This is a property test across the full SVData space.

Remark 27 (Eased segments). Quaver SSF and fluXis multiplier events relax piecewise-constant v to piecewise-smooth (linear, sinusoidal, etc.). The framework is unchanged: the integrator must compute per-segment closed-form antiderivatives instead of the constant case in Proposition 25. Numerical exactness becomes exactness up to the analytic accuracy of the easing-curve antiderivative.

9 Architectural mapping

The structural payoff of the measure-theoretic framing is that the engine collapses to a single integrator parameterized by the document’s measure. Engine identity reduces to a choice of μ .

9.1 The integrator

A single procedure handles all engines:

$$\text{cumulative_at}(t, \text{doc}) = \int_{(t_0, t]} v(\tau) d\mu_{\text{doc}}(\tau).$$

The procedure decomposes the integral by segments of the common refinement (Proposition 25) and accumulates AC contributions and atoms separately. `distance`, `render_multiplier_at`, and `inverse_cumulative_at` are derived from this primitive.

9.2 SVData

The canonical document carries:

- μ : the integration measure, stored as $(\rho(\tau) d\tau, \{(\tau_w, a_w)\})$ — a piecewise-constant AC density plus a list of atoms. For time-space-authored charts, $\rho \equiv 1$ and the atom list is empty. For beat-space-authored charts, $\rho = B'$ and atoms are warps.
- Velocity records: piecewise-constant v with selectors per §7. Each record carries $(\tau, b, m, \text{sel}, \text{group})$; the engine uses τ for time-space evaluation, b for beat-space.
- Speed curves z : position-dependent zoom, optional, with per-segment easing.
- Time offsets: per-(time, group) animated visual offsets, optional.
- Coupling flag: BPM-coupled vs. BPM-decoupled (§6).

By Corollary 19, the canonical form supports atoms in μ ; time-space-authored charts embed losslessly by leaving the atom list empty.

9.3 Engines as measure choices

Each engine corresponds to a fixed μ :

Engine	μ	atoms	z	warps	per-col
TimeSpace	$d\tau$	dropped	opt	dropped	opt
BeatSpace	dB	honored	yes	native	opt
ConstantBPM	$dB, v \equiv 1$	honored	const	dropped	no
Identity	$d\tau, v \equiv 1$	—	no	—	no

The TimeSpace and BeatSpace engines run *the same integrator* on the same SVData; they differ only in the μ they request. ConstantBPM additionally pins $v \equiv 1$ and z to the player’s hi-speed. Identity pins both to constants.

9.4 Engine swap

Per Remark 16, swap reduces to:

1. Reconfigure the integrator’s measure to the new engine’s μ .
2. Discard C_μ and rebuild from the document.
3. Re-anchor the clock smoother by mapping the current chart-time t forward through the new C_μ , not by translating the old value.
4. Refresh `sv_enabled` from the new engine’s flag; if a CMOD-style override is suspended, refresh that state too.

The renderer’s per-frame contract is unchanged.

10 Per-frame predictor (cull-space)

The renderer queries $C(t)$ every frame to position notes in cull-space. Two correctness criteria must be met:

Exactness on audio callback. When the audio engine delivers a fresh chart-time t_{raw} , the rendered cumulative must equal $C(t_{\text{raw}})$.

Smoothness between callbacks. Between callbacks, the rendered cumulative must advance linearly in wall-time at the local rate $r(t) = v(t)w(t)$, without observing per-frame noise from the audio engine’s source clock.

The naive renderer evaluates $C(t_{\text{raw}})$ per frame. This satisfies exactness but fails smoothness: t_{raw} inherits sub-callback stream-clock jitter on every per-frame read, producing output position noise of order $r \cdot \sigma_{\text{clock}}$, visible at high local rates.

10.1 The anchor-and-extrapolate algorithm

Maintain a four-tuple of state:

$$(t_a, C_a, r_a, W_a)$$

where t_a is the chart-time at the most recent anchor, $C_a = C(t_a)$ exactly, $r_a = r(t_a^+)$ from the engine’s `cumulative_velocity_at`, and W_a is wall-time (`monotonic()`) at the anchor instant.

Definition 28 (Per-frame query). At wall-time W with playback rate ρ (chart-seconds per wall-second), the predicted cumulative is

$$\tilde{C}(W) = C_a + r_a \cdot (W - W_a) \cdot \rho. \quad (7)$$

Theorem 29 (Equivalence on a constant-rate segment). *On a segment where r is constant on $[t_a, t_a + (W - W_a)\rho]$ and no warp atom intervenes, $\tilde{C}(W) = C(t_a + (W - W_a)\rho)$ exactly.*

Proof. On a constant-rate interval, $C(t) - C(t_a) = \int_{t_a}^t r d\tau = r \cdot (t - t_a)$. Substituting $t = t_a + (W - W_a)\rho$ gives $C(t) = C_a + r \cdot (W - W_a)\rho$, which equals (7). $\square$

10.2 Re-anchoring on breakpoints

When the predicted chart-time $t_a + (W - W_a)\rho$ would cross the next breakpoint b , the algorithm advances the anchor to that breakpoint:

$$(t'_a, C'_a, r'_a, W'_a) = (b, C(b), r(b^+), W_a + (b - t_a)/\rho).$$

$C(b)$ comes from the integrator (exact, including any warp atom at b); $r(b^+)$ is the post-boundary local rate. Subsequent per-frame queries extrapolate from this new anchor. Multiple breakpoints in one frame are handled by iterating until the predicted chart-time falls within the current cell.

Theorem 30 (Equivalence across breakpoints). *For any W and any sequence of breakpoints $b_1 < b_2 < \dots < b_n$ in $(t_a, t_a + (W - W_a)\rho]$, after n re-anchor iterations, $\tilde{C}(W) = C(t_a + (W - W_a)\rho)$.*

Proof. By induction on n . The base case $n = 0$ is Theorem 29. Inductive step: after re-anchoring at b_n , the predictor's state is $(b_n, C(b_n), r(b_n^+), W_a + (b_n - t_a)/\rho)$. The remaining extrapolation crosses no further breakpoints in the current frame (otherwise the loop continues), so by Theorem 29 the final value is $C(b_n) + r(b_n^+) \cdot (\text{remaining chart-time}) = C(t_{\text{end}})$, where the second equality holds because the integrator splits its grid at b_n and integrates exactly across each sub-segment. $\square$

10.3 Re-anchoring on audio callbacks and discontinuities

Audio callbacks update t_{raw} ahead of the predictor's extrapolation. The predictor detects this by comparing $t_{\text{raw}} - t_a$ against the predicted chart-time advance $(W - W_a)\rho$:

$$|(t_{\text{raw}} - t_a) - (W - W_a)\rho| > \theta \implies \text{re-anchor at } t_{\text{raw}}.$$

The threshold θ is chosen larger than typical sub-callback clock jitter ($\theta = 5$ ms in the implementation) so per-frame stream-clock noise does not trigger spurious re-anchors. Backward jumps in t_{raw} (seek, scrub release) trigger immediate re-anchor; explicit `reset(t)` calls (engine swap, rate change) re-anchor unconditionally.

10.4 Eased segments

For an eased cell where $v(\tau)$ is linear from v_s at $\tau = b_k$ to v_e at $\tau = b_{k+1}$, the integrator computes the per-cell contribution exactly via the trapezoid rule:

$$\int_{b_k}^{b_{k+1}} v w d\tau = \frac{1}{2}(v_s + v_e) \cdot w \cdot (b_{k+1} - b_k).$$

Within the cell, the integrator’s $C(t)$ is quadratic in $(t - b_k)$. The predictor’s linear extrapolation at $r_a = v_s \cdot w$ (start-of-cell rate) is therefore inexact within the cell, with worst-case error

$$|\tilde{C}(t) - C(t)| \leq \frac{1}{2}|v_e - v_s| \cdot w \cdot (b_{k+1} - b_k),$$

attained at the cell midpoint. The error vanishes at the cell endpoints, where the predictor re-anchors. This is acceptable in practice because authored eased cells are short ($\sim$ ms scale for SSF / fluXis events), and re-anchor at the cell endpoint zeroes the error before it propagates.

10.5 Performance and jitter trade

Measured with 50 SV breakpoints across 600 s of chart, in the Python implementation:

- Per-frame $C(t_{\text{raw}})$: $\sim 1.0 \mu\text{s}$.
- Predictor: $\sim 2.9 \mu\text{s}$.

At 60 Hz, the predictor adds $\sim 3 \mu\text{s}/\text{frame} - 0.018\%$ of the frame budget.

Jitter reduction, measured by frame-to-frame ΔC standard deviation under Gaussian noise on t_{raw} with constant rate:

σ_{in} (ms RMS)	0.10	0.50	1.00
ΔC stddev (no predictor)	0.000435	0.001448	0.002841
ΔC stddev (predictor)	0.000331	0.000320	0.000573
ratio	1.3 $\times$	4.5 $\times$	5.0 $\times$

In the realistic regime ($\sigma_{\text{in}} \approx 0.5\text{--}1$ ms for PortAudio’s `stream.time`), the predictor reduces visible jitter by $\sim 5\times$. At 60 Hz with `scroll_speed = 500` px/s, this turns ~ 1 pixel of frame-to-frame jitter into ~ 0.16 pixels – below visual perception threshold for most viewers.

10.6 Native-port boundary

The predictor and integrator are pure-functional given their inputs (no Player-state coupling, no thread state). The fast path of $\tilde{C}$ in native code is one `monotonic()` syscall, ~ 4 floating-point operations, and two branches – estimated $\sim 10\text{--}20$ ns per frame in C / Rust / Zig. The Python implementation’s $\sim 3 \mu\text{s}$ overhead is interpreter cost; lifting the math to a native module behind an FFI shim would make the perf trade fully favorable.

The state struct, breakpoint cursor, and re-anchor protocol map to a native module without re-deriving anything; the boundary is marked in the source files.

11 Open extensions

- **Per-note SV**: formally supported by Theorem 23; deferred pending sparse cumulative storage.
- **Modscripting**: adds a per-frame mutation of v , z , and `sel`. Out of scope; the framework’s static-positioning subset is sufficient for non-modfile charts.
- **Eased SV segments**: Proposition 25 relaxes to per-segment closed-form antiderivatives; Remark on numerical exactness applies.
- **Singular non-atomic measures**: the framework permits μ_{sing} that is non-atomic (e.g. Cantor-like). No game uses these; the framework would handle them in principle but no engine implements such measures.